

Cardiac Patient Monitoring System API

Overview The project is a standalone ASP.NET Core REST API designed as a cardiac monitoring training prototype. The application manages patient records, vital signs, medications, and appointments. All records in the project are explicitly synthetic and should not be treated as clinical data.

Project Objectives - Manage patient information - Track vital signs such as heart rate and blood pressure - Record medications and treatment plans - Schedule and track appointments with clinicians - Secure API access using JWT authentication - Provide a documented API using Swagger/OpenAPI

Technology Stack - ASP.NET Core 10 - Entity Framework Core - SQL Server - ASP.NET Core Identity - JWT Authentication - Swagger / OpenAPI - xUnit and Moq

Project Structure The solution contains an API project and a test project. The main API project includes the following folders: - Controllers: request handling and route endpoints - Models: core entities such as Patient, VitalSign, Medication, and Appointment - DTOs: request and response data contracts - Data: DbContext and EF Core configuration - Services: business logic for authentication, patients, vital signs, medication, and appointments - Middleware: centralized exception handling - Migrations: database evolution files - Tests: automated verification for controllers and middleware

Core Entities Patient: stores patient identity information such as first name, last name, date of birth, gender, and phone number. VitalSign: stores heart rate, systolic blood pressure, diastolic blood pressure, temperature, and recorded time. Medication: stores drug name, dosage, frequency, start date, and optional end date. Appointment: stores scheduled time, clinician name, reason, and status. ApplicationUser: represents the identity user used for authentication.

Authentication and Authorization The API uses ASP.NET Core Identity together with JWT authentication. The authentication flow includes user registration and login endpoints, both returning a JWT token. Protected routes require the token in the Authorization header using the Bearer scheme.

Main API Endpoints Auth - POST /api/auth/register - POST /api/auth/login

Patients - GET /api/patients - GET /api/patients/{id} - POST /api/patients - PUT /api/patients/{id} - DELETE /api/patients/{id}

Vital Signs - GET /api/patients/{patientId}/vital-signs - GET /api/vital-signs/{id} - POST /api/patients/{patientId}/vital-signs - PUT /api/vital-signs/{id} - DELETE /api/vital-signs/{id}

Medications - GET /api/patients/{patientId}/medications - GET /api/medications/{id} - POST /api/patients/{patientId}/medications - PUT /api/medications/{id} - DELETE /api/medications/{id}

Appointments - GET /api/patients/{patientId}/appointments - GET /api/appointments/{id} - POST /api/patients/{patientId}/appointments - PUT /api/appointments/{id} - DELETE /api/appointments/{id}

Request Workflow A client request begins in a controller, which receives the HTTP request and binds the body to a DTO. The controller calls the appropriate service, where validation and business logic are executed. The service then interacts with ApplicationDbContext, which manages EF Core database access and persists entities to SQL Server. Finally, a response DTO is returned to the client.

Example: creating a patient 1. The client sends a POST request to /api/patients with patient data. 2.

PatientsController receives the request and model binds the data to PatientRequestDto. 3. The controller calls PatientService.CreateAsync(dto). 4. The service validates the date of birth and creates a Patient entity. 5. The DbContext adds the entity and saves the changes to the database. 6. The API returns a PatientResponseDto representing the newly created patient.

Error Handling and Validation Input validation is enforced using data annotations in DTOs. Invalid requests are rejected with validation errors. Central middleware handles exceptions and returns safe JSON error responses for missing resources and unexpected failures.

Testing and Validation The project includes automated tests using xUnit and Moq. The validation command was executed successfully: Command: dotnet test "CardiacPatientMonitoring.slnx" --nologo Result: 11 tests passed, 0 failed. This confirms that the project builds successfully and the current automated test suite passes.

Conclusion The Cardiac Patient Monitoring System API provides a practical example of a secure ASP.NET Core backend for healthcare-related data management. It combines a clean layered architecture, database integration, authentication, validation, and API documentation in a compact and maintainable solution. The project is suitable as an academic or training prototype and is ready for further extension and deployment.